

VetNova

Guia de Defensa Tecnica

Mis 3 microservicios: [ficha \(8087\)](#) · [catalogo \(8082\)](#) · [agenda \(8086\)](#)

■ Info importante

- Sucursales validas: CHILLAN · LOS_ANGELES · TALCA
- Guardar el id de cada respuesta para usarlo en pasos siguientes
- WebClient (NO RestTemplate) para llamadas entre microservicios
- agenda llama a auth (8081) y a ficha (8087) — son las UNICAS conexiones

Contenido

Sección	Contenido
1	Mapa de conexiones entre mis 3 MS
2	Endpoints 'escondidos' — query params y URLs especiales
3	Flujo principal — Agendar una cita (12 pasos)
4	Flujo ficha clinica — inmutabilidad y soft delete
5	Flujo catalogo — buscar, activar, precio
6	Preguntas de defensa con respuestas

1. Mapa de conexiones entre mis 3 MS

Quien llama a quien

```
+-----+
| vetnova_auth (8081) |
| Login, usuarios, verificar cliente |
+-----+
| WebClient
| GET /api/usuarios/{id}/existe <- validacion DURA
| GET /api/usuarios/{id} <- enriquecimiento SUAVE
v
+-----+
| vetnova_agenda (8086) |
| Citas, boxes, disponibilidad |
+-----+
| WebClient
| GET /api/v1/mascotas/{id} <- validacion DURA
| GET /api/v1/mascotas/{id} <- enriquecimiento SUAVE
v
+-----+
| vetnova_ficha (8087) |
| Mascotas, fichas, historial medico |
+-----+
+-----+
| vetnova_catalogo (8082) |
| Productos, servicios, categorias |
| (AUTONOMO – no llama a ningun otro MS) |
+-----+
```

Detalle de las 2 conexiones de agenda

agenda -> auth: verificar cliente

```
1. Verificacion DURA (si falla -> cita NO se crea):
GET http://localhost:8081/api/usuarios/{clienteId}/existe
-> { "existe": true }
-> Si false o error -> HTTP 404 "Cliente no encontrado"

2. Enriquecimiento SUAVE (si falla -> cita igual se crea):
GET http://localhost:8081/api/usuarios/{clienteId}
-> { "nombre": "Luis", "apellido": "Hernandez" }
-> Retorna: "Luis Hernandez" como nombreCliente
-> Si falla -> nombreCliente = null (la cita igual se guarda)
```

agenda -> ficha: verificar mascota

```
1. Verificacion DURA (si falla -> cita NO se crea):  
GET http://localhost:8087/api/v1/mascotas/{mascotaId}  
-> Si error -> HTTP 404 "Mascota no encontrada"  
  
2. Enriquecimiento SUAVE (si falla -> cita igual se crea):  
GET http://localhost:8087/api/v1/mascotas/{mascotaId}  
-> { "nombre": "Max", "especie": "Perro" }  
-> Retorna: "Max" como nombreMascota  
-> Si falla -> nombreMascota = null
```

Que pasa si un MS cae

MS caido	Impacto
auth (8081)	agenda NO puede crear citas. GET /citas muestra nombreCliente: null
ficha (8087)	agenda NO puede crear citas. GET /citas muestra nombreMascota: null
catalogo (8082)	No afecta a ningun otro MS. Es completamente independiente
agenda (8086)	No afecta a ficha ni a catalogo

2. Endpoints 'escondidos' — los que no son obvios

Estos endpoints tienen query params en la URL o son sub-rutas especiales. El profe puede preguntar por ellos específicamente porque no aparecen en la lista básica de endpoints.

vetnova_catalogo (8082) — query params obligatorios y opcionales

Endpoint	Query Params	Ejemplo
GET /api/v1/catalogo/disponibles	sucursal (OBLIGATORIO)	?sucursal=CHILLAN
GET /api/v1/catalogo/buscar	nombre	?nombre=Purina
GET /api/v1/catalogo/buscar/rango	min, max (NO precioMin/precioMax)	?min=1000&max;=50000
GET /api/v1/catalogo/buscar/categoria	categoriald	?categoriald=1
GET /api/v1/catalogo/detalle	itemId, tipo (NO sucursal)	?itemId=1&tipo;=producto
PUT /api/v1/productos/{id}/precio	nuevoPrecio (en URL, NO en body)	?nuevoPrecio=9990
PUT /api/v1/servicios/{id}/precio	nuevoPrecio (en URL, NO en body)	?nuevoPrecio=25000

vetnova_catalogo — activar/desactivar y precio (con la URL correcta)

```
PUT http://localhost:8082/api/v1/productos/{id}/activar
-> activo = true (el producto aparece en el catalogo)
PUT http://localhost:8082/api/v1/productos/{id}/desactivar
-> activo = false (no aparece pero NO se borra de la BD)
PUT http://localhost:8082/api/v1/productos/{id}/precio?nuevoPrecio=39990
-> OJO: la subruta es /precio y el valor va en query param
PUT http://localhost:8082/api/v1/servicios/{id}/activar
PUT http://localhost:8082/api/v1/servicios/{id}/desactivar
PUT http://localhost:8082/api/v1/servicios/{id}/precio?nuevoPrecio=22000
```

vetnova_agenda (8086) — endpoints especiales

Endpoint	Que hace
GET /api/v1/citas/agenda	Citas de HOY enriquecidas con nombres (NO es /hoy)
GET /api/v1/citas	Todas las citas con nombres de cliente y mascota
PUT /api/v1/citas/{id}	Reprogramar (body: fechaHora + duracionMinutos)
PUT /api/v1/citas/{id}/confirmar	PENDIENTE -> CONFIRMADA
PUT /api/v1/citas/{id}/iniciar	CONFIRMADA -> EN_CURSO
PUT /api/v1/citas/{id}/completar	EN_CURSO -> COMPLETADA
PUT /api/v1/citas/{id}/cancelar	Cualquier estado -> CANCELADA (body: motivoCancelacion)
PUT /api/v1/boxes/{id}/liberar	Cambia box a DISPONIBLE (tras terminar cita)

Endpoint	Que hace
PUT /api/v1/boxes/{id}/reservar	Cambia box a NO DISPONIBLE (OJO: es /reservar, NO /ocupar)

vetnova_ficha (8087) — soft delete, enriquecimiento y query params escondidos

```

DELETE http://localhost:8087/api/v1/mascotas/{id}
-> NO borra de la BD -> pone activo = false
-> Verificar: respuesta 200 con activo = false
GET http://localhost:8087/api/v1/mascotas/{id}
-> Retorna campo nombreCliente (obtenido desde auth por WebClient)
-> Si auth esta caido -> nombreCliente = null (degradacion suave)
--- Query params escondidos de ficha (filtrar por fichaId) ---
GET http://localhost:8087/api/v1/fichas?mascotaId=1
-> Retorna la ficha de esa mascota especifica
GET http://localhost:8087/api/v1/evoluciones?fichaId=1
GET http://localhost:8087/api/v1/vacunas?fichaId=1
GET http://localhost:8087/api/v1/recetas?fichaId=1
GET http://localhost:8087/api/v1/procedimientos?fichaId=1
GET http://localhost:8087/api/v1/certificados?fichaId=1
Inmutables – HTTP 405 (Method Not Allowed) si se intenta editar o borrar:
PUT /api/v1/evoluciones/{id} -> 405 SIEMPRE
DELETE /api/v1/evoluciones/{id} -> 405 SIEMPRE
PUT /api/v1/vacunas/{id} -> 405 SIEMPRE
PUT /api/v1/recetas/{id} -> 405 SIEMPRE
PUT /api/v1/certificados/{id} -> 405 SIEMPRE

```

3. Flujo principal — Agendar una cita (12 pasos)

Este flujo toca los 3 microservicios del alumno.

Paso 1 — Login (auth, 8081)

POST <http://localhost:8081/api/auth/login>

```
{
  "email": "repcionista@vetnova.cl",
  "password": "1234"
}
```

Verificar: 200 · campo token presente

Paso 2 — Ver catalogo de servicios (catalogo, 8082)

GET <http://localhost:8082/api/v1/catalogo/disponibles?sucursal=CHILLAN>

Verificar: 200 · lista de servicios activos · guardar servicioid

Paso 3 — Registrar mascota (ficha, 8087)

POST <http://localhost:8087/api/v1/mascotas>

```
{
  "clienteId": 1,
  "nombre": "Firulais",
  "especie": "Perro",
  "raza": "Labrador",
  "sexo": "Macho",
  "fechaNacimiento": "2020-03-15",
  "peso": 28.5,
  "activo": true
}
```

Verificar: 201 · campo id -> guardar como mascotaid

Paso 4 — Crear ficha clinica (ficha, 8087)

POST <http://localhost:8087/api/v1/fichas>

```
{
  "mascotaId": 1,
  "observacionesGenerales": "Primera visita, paciente sano"
}
```

Verificar: 201 · campo id -> guardar como fichaid

Paso 5 — Crear box (agenda, 8086)

POST <http://localhost:8086/api/v1/boxes>

```
{
  "nombre": "Box 1",
  "sucursal": "CHILLAN"
}
```

Verificar: 201 · campo id -> guardar como boxid

Paso 6 — Registrar disponibilidad veterinario (agenda, 8086)

POST <http://localhost:8086/api/v1/disponibilidad>

```
{
  "veterinarioId": 1,
  "sucursal": "CHILLAN",
  "diaSemana": "LUNES",
  "horaInicio": "08:00",
  "horaFin": "17:00"
}
```

Verificar: 201

Paso 7 — Crear la cita (agenda, 8086) — SE ACTIVAN LAS 2 LLAMADAS REMOTAS

POST <http://localhost:8086/api/v1/citas>

```
{
  "clienteId": 1,
  "mascotaId": 1,
  "veterinarioId": 1,
  "servicioId": 1,
  "boxId": 1,
  "sucursal": "CHILLAN",
  "fechaHora": "2030-08-15T10:00:00",
  "duracionMinutos": 30,
  "canal": "PRESENCIAL"
}
```

Internamente: WebClient llama a auth(8081) para verificar clienteld y a ficha(8087) para verificar mascotald

Verificar: 201 · estado = 'PENDIENTE' · campo id -> guardar como citaId

Paso 8 — Ver agenda del día (agenda, 8086)

GET <http://localhost:8086/api/v1/citas/agenda>

Verificar: 200 · campos nombreCliente, nombreMascota presentes

Paso 9 — Confirmar cita

PUT <http://localhost:8086/api/v1/citas/{citaId}/confirmar>

Verificar: 200 · estado = 'CONFIRMADA'

Paso 10 — Iniciar atencion

PUT <http://localhost:8086/api/v1/citas/{citaId}/iniciar>

Verificar: 200 · estado = 'EN_CURSO'

Paso 11 — Registrar evolucion (ficha, 8087) — INMUTABLE

POST <http://localhost:8087/api/v1/evoluciones>

```
{
  "fichaId": 1,
  "veterinarioId": 1,
  "citaId": 1,
  "anamnesis": "Dueno reporta decaimiento desde ayer",
  "diagnostico": "Sindrome febril agudo",
  "tratamiento": "Dipirona 500mg IM, reposo",
  "observaciones": "Control en 48 horas"
}
```

Verificar: 201 · queda registrado permanentemente, NO se puede editar ni borrar

Paso 12 — Completar cita y liberar box

PUT <http://localhost:8086/api/v1/citas/{citaId}/completar>

Verificar: 200 · estado = 'COMPLETADA'

PUT <http://localhost:8086/api/v1/boxes/{boxId}/liberar>

Verificar: 200 · estado = 'DISPONIBLE'

Errores importantes de este flujo

```
[ ] clienteId no existe en auth:
-> HTTP 404 "Cliente no encontrado" – cita NO se crea (validacion DURA)

[ ] mascotaId no existe en ficha:
-> HTTP 404 "Mascota no encontrada" – cita NO se crea (validacion DURA)

[ ] fechaHora en el pasado:
POST con "fechaHora": "2020-01-01T10:00:00"
-> HTTP 400 "La fecha y hora deben ser futuras"

[ ] sucursal invalida:
POST con "sucursal": "VALPARAISO"
-> HTTP 404 "Sucursal no encontrada"

Validas: CHILLAN, LOS_ANGELAS, TALCA
```

Reprogramar cita

```
PUT http://localhost:8086/api/v1/citas/{citaId}
Body: { "fechaHora": "2030-09-01T11:00:00", "duracionMinutos": 45 }

Validaciones internas:
[OK] La cita existe
[OK] Estado NO es COMPLETADA ni CANCELADA
[OK] Nueva fecha es futura
[OK] Veterinario no tiene otra cita en ese horario
```


4. Flujo ficha clinica — inmutabilidad y soft delete

Registrar historial completo de una mascota

Paso 1 — Vacuna (inmutable)

POST <http://localhost:8087/api/v1/vacunas>

```
{
  "fichaId": 1,
  "veterinarioId": 1,
  "nombre": "Antirrabica",
  "fechaAplicacion": "2026-06-30",
  "proximaFecha": "2027-06-30"
}
```

Verificar: 201 · una vez creada NO se puede editar ni eliminar

Paso 2 — Receta (inmutable)

POST <http://localhost:8087/api/v1/recetas>

```
{
  "fichaId": 1,
  "veterinarioId": 1,
  "medicamentos": [
    {
      "nombre": "Amoxicilina 500mg",
      "dosis": "1 comprimido",
      "frecuencia": "Cada 8 horas",
      "duracionDias": 7
    }
  ],
  "fechaVencimiento": "2026-07-30"
}
```

Verificar: 201 · inmutable

Paso 3 — Procedimiento (inmutable)

POST <http://localhost:8087/api/v1/procedimientos>

```
{
  "fichaId": 1,
  "veterinarioId": 1,
  "nombre": "Limpieza dental",
  "tipo": "PROFILAXIS",
  "descripcion": "Limpieza dental con ultrasonido",
  "fecha": "2026-06-30",
  "resultado": "Exitoso"
}
```

Verificar: 201 · inmutable

Paso 4 — Certificado (inmutable)

POST <http://localhost:8087/api/v1/certificados>

```
{
  "fichaId": 1,
  "veterinarioId": 1,
  "tipo": "SALUD",
  "descripcion": "El paciente se encuentra en buen estado de salud general."
}
```

Verificar: 201 · tipos validos: SALUD, VACUNACION, VIAJE, ADOPCION

Demostrar inmutabilidad (error esperado)

```
PUT http://localhost:8087/api/v1/evoluciones/{id}
Body: { "diagnostico": "Intentando editar" }
-> HTTP 405 METHOD_NOT_ALLOWED <- SIEMPRE da error (no 400, no 404)

DELETE http://localhost:8087/api/v1/evoluciones/{id}
-> HTTP 405 METHOD_NOT_ALLOWED <- SIEMPRE da error

Lo mismo aplica a: vacunas, recetas, procedimientos, certificados
Por que? Para no alterar el historial clinico legal del animal.
```

Demostrar soft delete (mascota)

```
DELETE http://localhost:8087/api/v1/mascotas/{id}
-> HTTP 200 · campo activo = false
-> La mascota NO se borra de la BD (sus fichas y registros siguen)

Por que? Para preservar el historial clinico completo del animal.
```

Intentar crear segunda ficha (error 409)

```
POST http://localhost:8087/api/v1/fichas
Body: { "mascotaId": 1 } <- mascotaId que ya tiene ficha
-> HTTP 409 "Ya existe una ficha clinica para esta mascota"

Una mascota solo puede tener UNA ficha clinica (relacion 1 a 1).
```

5. Flujo catalogo — buscar, activar, precio

Setup inicial

Paso 1 — Crear categoria

POST <http://localhost:8082/api/v1/categorias>

```
{
  "nombre": "Alimentos",
  "descripcion": "Alimentos y suplementos para mascotas",
  "tipo": "PRODUCTO"
}
```

Verificar: 201 · campo id -> guardar como categoriaId

Paso 2 — Crear producto

POST <http://localhost:8082/api/v1/productos>

```
{
  "nombre": "Purina Pro Plan 15kg",
  "descripcion": "Alimento premium para perros adultos",
  "precio": 45990.0,
  "categoriaId": 1
}
```

Verificar: 201 · campo id -> guardar como productId

Paso 3 — Crear servicio veterinario

POST <http://localhost:8082/api/v1/servicios>

```
{
  "nombre": "Consulta General",
  "descripcion": "Consulta medica veterinaria estandar",
  "precio": 25000.0,
  "duracionMinutos": 30,
  "categoriaId": 1
}
```

Verificar: 201 · campo id -> guardar como servicioId

Busquedas (endpoints escondidos)

```
GET http://localhost:8082/api/v1/catalogo/disponibles?sucursal=CHILLAN
-> Productos activos de esa sucursal (sucursal OBLIGATORIO)
GET http://localhost:8082/api/v1/catalogo/buscar?nombre=Purina
-> Busca por nombre (contiene, ignora mayusculas)
GET http://localhost:8082/api/v1/catalogo/buscar/rango?min=1000&max=50000
-> OJO: los params son 'min' y 'max' (NO precioMin/precioMax)
GET http://localhost:8082/api/v1/catalogo/buscar/categoria?categoriaId=1
-> Todos los items de esa categoria
GET http://localhost:8082/api/v1/catalogo/detalle?itemId=1&tipo=producto
-> OJO: params son 'itemId' y 'tipo' (NO sucursal); tipo en minuscula
-> tipo valido: 'producto' o 'servicio'
[ ] Sucursal invalida:
GET ...disponibles?sucursal=INVALIDA
-> HTTP 404 "Sucursal no encontrada"
```

Actualizar precio (subruta /precio + query param, NO body)

```
PUT http://localhost:8082/api/v1/productos/{productoId}/precio?nuevoPrecio=39990
-> HTTP 200 · precio actualizado
-> OJO: la URL tiene /precio al final, luego ?nuevoPrecio=X
-> NO va nada en el body
PUT http://localhost:8082/api/v1/servicios/{servicioId}/precio?nuevoPrecio=22000
-> HTTP 200 · precio actualizado
```

Activar/Desactivar (NO es DELETE)

```
PUT http://localhost:8082/api/v1/productos/{productoId}/desactivar
-> activo = false (desaparece del catalogo pero NO se borra)
PUT http://localhost:8082/api/v1/productos/{productoId}/activar
-> activo = true (vuelve a aparecer)
Lo mismo aplica a servicios:
PUT http://localhost:8082/api/v1/servicios/{servicioId}/desactivar
PUT http://localhost:8082/api/v1/servicios/{servicioId}/activar
```

Crear oferta

POST <http://localhost:8082/api/v1/ofertas>

```
{
  "productoId": 1,
  "descuento": 15.0,
  "fechaInicio": "2026-07-01",
  "fechaFin": "2026-07-31",
  "activa": true
}
```

Verificar: 201 · descuento en porcentaje (1-100)

6. Preguntas de defensa — con respuestas

¿Que es un Bounded Context?

Es el limite de responsabilidad de un microservicio. Por ejemplo, "Catalogo" solo sabe de productos, servicios y categorias — no sabe nada de clientes ni de citas. Cada MS tiene su propia base de datos y sus propias reglas. Si catalogo y ventas necesitan el mismo dato, cada uno lo guarda por separado en su BD. El limite esta muy claro: catalogo nunca tiene tablas de clientes ni de citas.

¿Que es el Gateway y para que sirve?

Es el punto de entrada unico al sistema (puerto 8080). En vez de que el cliente llame directo a localhost:8087, llama a localhost:8080 y el Gateway redirige al MS correcto segun la ruta. Su YAML de configuracion define: id (nombre de la ruta), uri (a donde redirige), predicates (que URLs captura). Permite centralizar seguridad y tener una sola URL publica para todos los MS.

Explica degradacion suave vs dura con ejemplos de tu codigo

Tipo	Que pasa	Ejemplo en el codigo
DURA	Si falla, operacion NO se completa	clienteld no existe en auth -> cita no se crea (HTTP 404)
DURA	Si falla, operacion NO se completa	mascotald no existe en ficha -> cita no se crea (HTTP 404)
SUAVE	Si falla, continua con datos incompletos	auth caido al obtener nombre -> cita se crea con nombreCliente = null
SUAVE	Si falla, continua con datos incompletos	ficha caida al obtener nombre -> cita se crea con nombreMascota = null

¿Por que los registros medicos son inmutables?

Porque son registros clinicos legales. Si un veterinario comete un error en la evolucion, debe crear una nueva evolucion corrigiendo — nunca editar la anterior. El historial clinico tiene que ser integro y auditable. En el codigo, los metodos PUT y DELETE de evoluciones, vacunas, recetas, procedimientos y certificados lanzan `RegistroImmutableException` con HTTP 400 siempre.

¿Por que usaron WebClient y no RestTemplate?

`WebClient` es el estandar actual de Spring para comunicacion entre microservicios — `RestTemplate` esta siendo deprecado. `WebClient` maneja mejor los errores remotos y nos permite implementar degradacion suave con `try/catch`. En el codigo, `AuthClient` y `FichaClient` usan `WebClient` con `.block()` para esperar la respuesta de forma sincrona. El `.block()` se usa porque el resto del sistema es MVC sincrono, no reactivo.

¿Que es un mock en los tests?

Un mock es un objeto falso que simula el comportamiento de uno real. En los tests de `CitaService` no llamamos de verdad a la BD ni a auth — creamos un mock del repositorio y del `AuthClient` que devuelven datos fijos. Asi el test es rapido, predecible y no depende de que los otros servicios esten corriendo.

¿Que es Given/When/Then?

Es la estructura de un test: Given (dado) preparo el escenario, When (cuando) ejecuto el metodo a probar, Then (entonces) verifico el resultado.

```
@Test
void crearCita_exitosa() {
    // GIVEN – preparo el escenario
    Cita cita = new Cita();
    cita.setClienteId(1L);
    cita.setMascotaId(1L);
    cita.setFechaHora(LocalDate.now().plusDays(1));
    when(authClient.usuarioExiste(1L)).thenReturn(true);
    when(citaRepository.save(any())).thenReturn(cita);

    // WHEN – ejecuto el metodo a probar
    Cita resultado = citaService.crear(cita);

    // THEN – verifico el resultado
    assertEquals("PENDIENTE", resultado.getEstado());
    verify(citaRepository, times(1)).save(any());
}
```

Test nuevo en vivo — fecha en el pasado

```
@Test
void crearCita_conFechaPasada_lanzaExcepcion() {
    // GIVEN
    Cita cita = new Cita();
    cita.setFechaHora(LocalDate.now().minusDays(1)); // pasado

    // WHEN / THEN
    assertThrows(IllegalArgumentException.class,
        () -> citaService.crear(cita));
}
```

Modificaciones en vivo mas probables

El profe pide	Que hacer	Donde
Agregar validacion	if (campo == null) throw new Exception("...")	Service, metodo crear()
Cambiar HTTP 200 a 201	ResponseEntity.status(HttpStatus.CREATED).body(...)	Controller
Agregar log	log.info("Evento: {}", valor)	Al inicio del metodo en el Service
Endpoint nuevo	@GetMapping + metodo en Controller + metodo en Service	Controller + Service

¿Por que 13 microservicios y no uno solo?

Con un monolito, si falla el modulo de reportes se cae todo. Con microservicios, si reportes falla, las citas y ventas siguen. Ademas cada MS se puede escalar por separado: si hay muchas ventas, escalo solo ventas sin tocar los demas. Y cada equipo puede trabajar en su MS sin pisar el trabajo de los otros.